泰康医养知识图谱智能问答系统——需求规格说明书

1. 业务与用户需求说明

1.1 详细业务背景与建设痛点

随着我国老龄化进程不断加快，“保险+医养”深度融合已成为大健康产业的必然趋势。然而，在当前的业务生态中，面临着以下核心痛点：

（1）知识维度广且割裂：该领域横跨“金融保险”、“临床医疗”与“养老服务”三个专业门槛极高的领域，数据散落在医保目录、保险条款（PDF 或文本）、民政部养老机构名单等不同来源中，存在严重的术语不统一如“高血压”与“原发性高血压”和“信息孤岛”现象。

（2）大模型的“误用”瓶颈：通用大语言模型（LLM）虽然具备强大的自然语言理解与生成能力，但在面对严谨的医疗、保险准入等专业问题时，极易产生“误用”（即生成看似合理但严重违背事实的错误回答），这在金融医疗领域是不可接受的风险。

1.2 核心建设目标与预期业务价值

本项目旨在引入 GraphRAG（**基于知识图谱的检索增强生成**）架构，解决上述痛点：

（1）构建跨域知识图谱底座：从至少两类以上数据源中抽取实体与关系，建立包含疾病、药品、保险产品、养老机构等实体（规模≥500 实体、≥1000 关系）的底层知识网络。

（2）实现精准可信的问答：利用图数据库（Neo4j）的确定性保障事实准确，利用大模型（Qwen/DeepSeek）的推理能力生成易懂回答，实现回答的“准确、可读、可溯源”。

1.3 详细用户故事（User Story）与验收标准

为了全方位覆盖业务场景，我们将用户划分为“C 端需求用户”与“B 端系统管理员”，并定义以下核心用户故事：

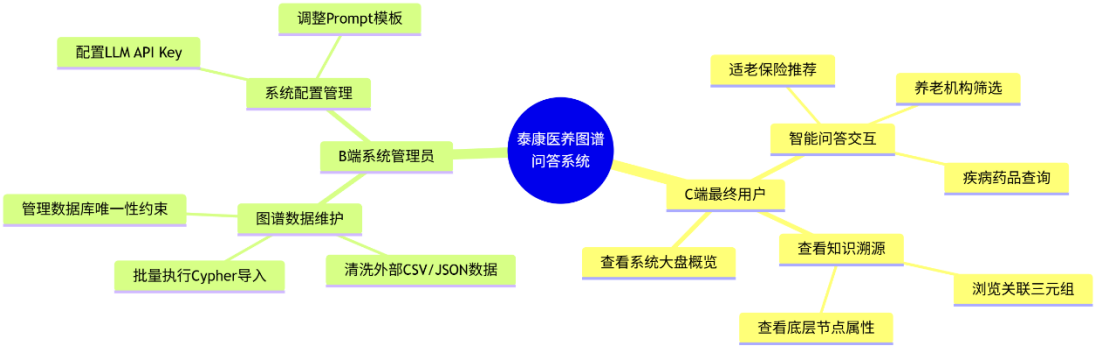

图 1.1 系统核心用例图

（1）US-01：基于健康画像的适老保险智能推荐

①作为一名老年用户或其家属，

②我希望能够使用口语化的自然语言提问（例如：“70 岁、有高血压，能买什么护理险或防癌险？”），

③以便于系统能自动校验我的年龄限制和既往病史，精准推荐我可以投保的产品。

④验收标准：系统必须准确提取问题中的年龄（70 岁）、疾病（高血压）、意图（保险查询）。推荐结果必须严格过滤掉准入年龄低于 70 岁或健康告知明确拒保高血压的产品。同类推荐结果需具备随机打乱机制（ORDER BY rand()），保证多次提问的推荐多样性。

（2）US-02：结构化养老机构动态筛选

①作为有养老安置需求的用户，

②我希望能够根据城市、价格区间等条件进行筛选（例如：“北京价格 5000 元以下的养老院有哪些？”），

③以便于快速锁定符合家庭预算的养老院。

④验收标准：系统能解析价格与地域意图，在图谱中进行属性过滤，并返回机构名称及具体地址。

（3）US-03：医疗与药品知识的多跳追踪

①作为关注健康状况的用户，

②我希望查询某种疾病（如糖尿病）的并发症以及对症的治疗药品，

③以便于获得综合的医疗科普参考。

④验收标准：系统需在图谱中完成“1-2 跳”的多级查询（疾病 -> 查找并发症 -> 查找对应药品），并将复杂的医疗三元组转化为易懂的白话解答。

（4）US-04：答案溯源与透明度展示

①作为严谨的系统使用者，

②我希望在 AI 给出建议时，能够明确看到它参考了哪些底层数据，

③以便于我信任该回答的权威性。

④验收标准：前端 UI 在展示回答的同时，必须附带展示或高亮引用的图谱节点信息（Reference）和来源三元组。

1.4 核心业务全链路流程分析

系统完整业务流可拆解为以下精密步骤：

（1）自然语言输入：用户通过 Streamlit 适老化前端侧边栏或主界面输入问题。

（2）意图与实体识别 (Query Understanding)：调用 LLM（如 DeepSeek）分析上下文，输出结构化的 JSON（如提取 intent, disease, age 等）。

（3）图谱检索构造 (Graph Retrieval)：检索器根据意图，动态拼装 Cypher 查询语句（例如 MATCH (p:Insurance)-[r:COVERS_DISEASE]->(d:Disease) WHERE d.name='高血压' RETURN p）。

（4）子图匹配与提取：连接 Neo4j 数据库，执行 Cypher，提取 1-2 跳范围内的关联三元组及节点属性。

（5）上下文组装 (Context Construction)：将检索到的图谱数据（JSON 或列表形式）格式化为自然语言描述段落。

（6）Prompt 工程增强：将“用户原始问题”+“图谱上下文”注入系统预设的“保险+医养专家”System Prompt 中。

（7）大模型生成：请求云端 API（如阿里云百炼 Qwen 模型）或本地小模型（如 Qwen-1.8B-Chat）。

（8）流式响应与溯源渲染：前端类似标准打字机效果输出最终答案，并在卡片下方展示 Reference 数据源。

2. 系统需求分析与建模

2.1 系统用例图说明 (Use Case Specification)

系统参与者：C 端最终用户 (End User)；B 端知识库管理员 (KG Admin)。

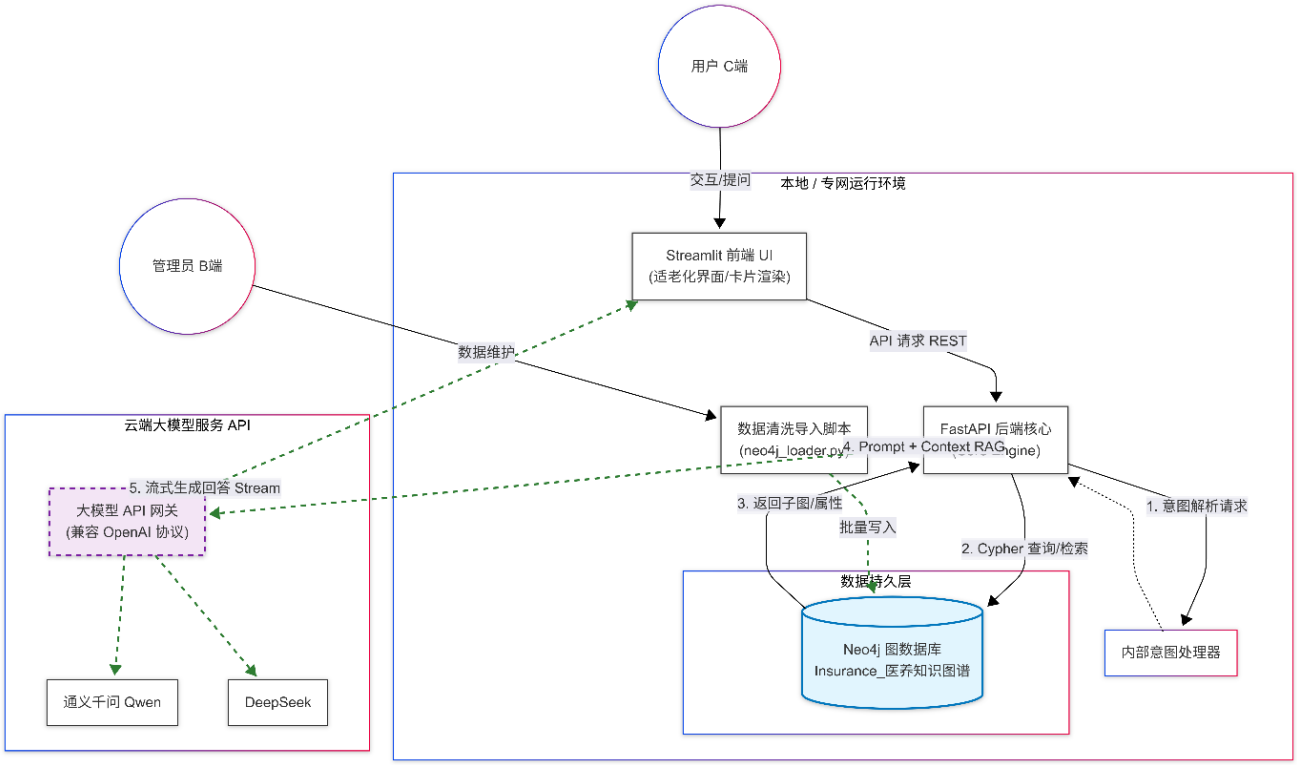

图 2.1 系统用例图说明图

表 1 用例/参与者和前后置条件说明表

用例名称	参与者	用例描述与主干前置/后置条件
执行智能对话	最终用户	**前置条件：**系统后端服务正常运行，Neo4j 及 LLM API 连通。 **主成功场景：**用户提问 -> 系统解析意图 -> 检索图谱 -> LLM 融合生成回答 -> 界面展示解答及来源引用。
查看知识溯源	最终用户	**场景：**用户点击 AI 回复下方的“溯源信息”卡片，查看支撑该回答的具体节点属性和关联三元组。

全局图谱概览	最终用户	**场景：**通过前端侧边栏实时查看当前图数据库中疾病、保险、养老院等节点的宏观数量统计图表。
批量导入实体数据	KG 管理员	**主成功场景：**管理员将清洗好的 JSON/CSV 文件交由 neo4j_loader.py 脚本处理，系统执行 UNWIND 语法批量将节点及关系写入图数据库，并自动更新唯一性约束（Unique Constraints）防止重复。

2.2 核心问答业务时序模型（Sequence details）

完整的 RAG 问答交互生命周期如下：

（1）End User -> Streamlit Frontend: 发送查询（例：“糖尿病买什么险？”）。

（2）Streamlit Frontend -> FastAPI Backend: 发送 POST /chat 请求，携带问题字符串。

（3）FastAPI Backend -> Query Understanding (LLM): 请求解析意图。

（4）Query Understanding (LLM) --> FastAPI Backend: 返回解析结果：{"intent": "insurance_query", "entities": {"Disease": ["糖尿病"]}}。

（5）FastAPI Backend -> Graph Retriever: 传递结构化参数，请求相关知识。

（6）Graph Retriever -> Neo4j Database: 动态执行 Cypher 子图查询。*[异常处理分支]*：若 Neo4j 查询结果为空（未找到匹配实体），系统捕获异常，生成“未找到相关精确知识”的兜底 Context。

（7）Neo4j Database --> Graph Retriever: 返回 [Insurance A, COVERS, 糖尿病], [Insurance B, COVERS, 糖尿病]。

（8）Graph Retriever --> RAG Engine: 将三元组拼装为 Context 文本。

（9）RAG Engine -> LLM Integration (Qwen/DeepSeek): 结合预设 Prompt（融入安全过滤要求）和 Context 发起流式推理请求。

（10）LLM Integration --> Streamlit Frontend -> End User: 持续推流回答文本，并在结束时附带溯源元数据。

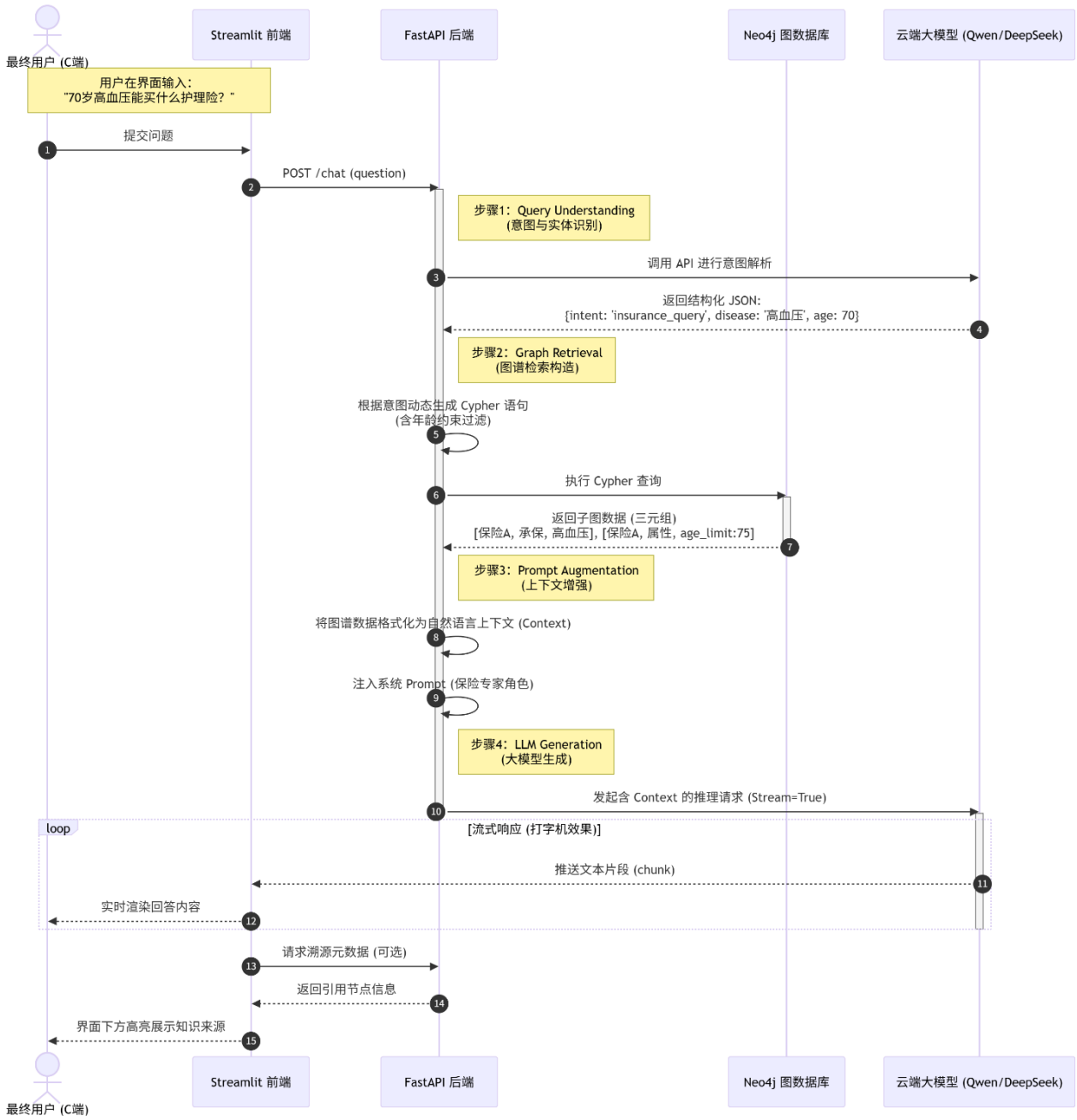

图 2.2 回答问题全链路时序说明图

2.3 图谱数据建模与详细数据字典 (Graph Schema & Data Dictionary)

本项目改变传统关系型数据库的二维表设计，采用“节点-关系-节点”的图数据模型（Property Graph Model）。

2.3.1 节点概念实体 (Nodes)

包含以下核心类别（Labels）：

（1）Disease（疾病）：疾病主体概念。

（2）Drug（药品）：用于治疗疾病的药物载体。

（3）Symptom（症状）：患者表现出的临床体征。

（4）NursingHome（养老机构）：提供线下养老照护服务的实体机构。

（5）Insurance（保险）：泰康及其他商业保险产品。

（6）Department（科室）：医院的行政与医疗划分科室。

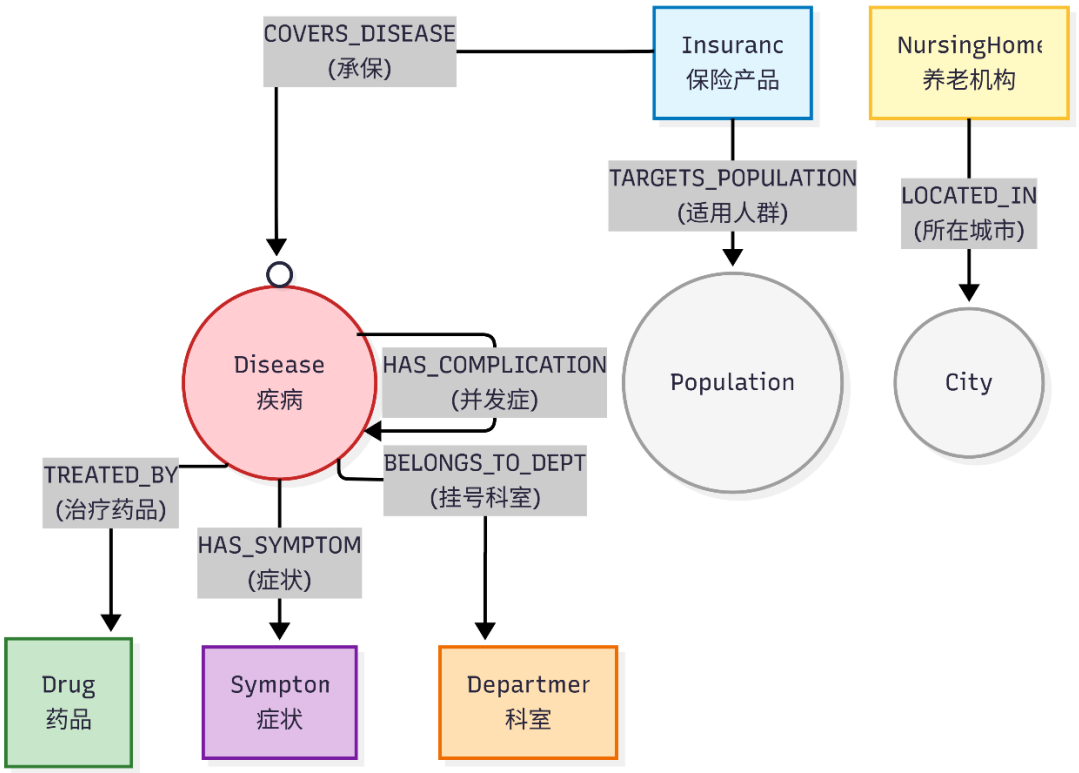

图 2.3 节点概念说明图

2.3.2 图谱边缘关系（Relationships）

定义实体间的有向边（Edges），支持 1-2 跳的多级遍历：

（1）(Disease)-[:HAS_SYMPTOM]->(Symptom)：疾病具有的症状。

（2）(Disease)-[:TREATED_BY]->(Drug)：疾病可通过某药品治疗。

（3）(Disease)-[:BELONGS_TO_DEPT]->(Department)：疾病挂号所属科室。

（4）(Disease)-[:HAS_COMPLICATION]->(Disease)：疾病可能引发的并发症。

（5）(Insurance)-[:COVERS_DISEASE]->(Disease)：保险产品覆盖或承保的疾病类型。

（6）(Insurance)-[:TARGETS_POPULATION]->(Population)：保险适用的人群群体。

2.3.3 核心数据字典（节点属性）

表 2 节点属性说明表

节点标签 (Label)	属性名称 (Property)	数据类型	说明/示例
Disease	name	String	主键约束，如“原发性高血压”
	icd_code	String	国际疾病分类代码（如 E11.9 糖尿病）
	treatment_details	String	临床治疗手段说明
Drug	name	String	药品通用名称
	specification	String	药品包装或剂量规格
NursingHome	name	String	养老机构全称

	city	String	机构所在城市（用于精准地域检索）
	price	Integer/String	月基础费用或价格区间（用于预算筛选）
	address	String	机构详细物理地址
Insurance	name	String	保险产品官方名称
	age_limit	Integer/Array	允许投保的最高年龄或年龄区间阈值
	target_population	String	适用人群画像（如“三高人群适用”、“老年防癌专项”）

3. 非功能性需求说明

3.1 详细系统架构与运行环境需求

本项目采用完全的前后端分离与微服务化架构思想设计：

（1）核心开发语言：Python 3.10 或 3.11。

（2）底层图数据库环境：Neo4j Desktop 或企业版 Server，服务需开放默认的 bolt://localhost:7687 端口进行通信。

（3）后端 API 框架：FastAPI。需要通过 uvicorn src.api.main:app --reload 部署，对外提供高性能的异步 RESTful 服务接口（如/chat）。

（4）前端交互 UI：Streamlit。进行 Custom CSS 优化以实现适老化视觉大字体、卡片式交互排版。

（5）大模型运行环境：云端 API 模式：接入阿里云百炼平台（调用通义千问 Qwen-turbo/max）或基于 OpenAI 兼容协议接入 DeepSeek，适合常规计算平台。**本地私有化部署：**针对高度敏感的医疗数据，系统预留了支持加载开源小模型（如 Qwen-1.8B-Chat 4-bit 量化版），需配备消费级及以上独立 GPU 环境。

3.2 性能与前端容错需求

（1）API 响应时效：对于涉及外部云端大模型的流式推理，后端到前端的首字输出延迟（TTFT）应控制在 1.5-2 秒内。

（2）图数据库查询效率：1-2 跳的 Cypher 关联检索时间应保持在毫秒级（< 200ms），这要求在图谱构建期间必须对 Disease.name、Insurance.name 等核心检索字段建立数据库索引（Indexes）。

（3）UI 容错渲染机制：当前端发起问答请求但图谱检索结果为空白（如冷门疾病无数据）时，前端框架代码必须能够捕获处理该状态，防止如 st.columns argument must be positive integer 此类的组件级致命渲染报错导致页面崩溃。

3.3 安全性、合规性与隐私需求

医疗与金融属性决定了系统必须具备极高的安全防护标准：

（1）绝对秘钥隔离（Environment Variables）：严禁在代码仓库（如 GitHub）或普通的 config.yaml 配置文件中硬编码敏感信息。所有如 DASHSCOPE_API_KEY（大模型调用凭证）与 NEO4J_PASSWORD（数据库密码）必须统一抽离，存储于本地项目根

目录下的.env 文件中，并通过 python-dotenv 动态加载，且.env 文件必须被配置在.gitignore 名单中拦截上传。

（2）公共配置统一管理：脱敏的应用层配置（如模型名称配置 qwen-turbo、数据源相对路径、Neo4j 本地通信 URI）使用公共的 config.yaml 结合 YAML 解析器维护，以支持开发、测试环境的快速切换。

（3）防注入与数据保护：构建动态 Cypher 语句时需使用参数化查询（Parameterized Queries），防止恶意的图数据库注入攻击（Cypher Injection）。针对养老机构与保险原始导入数据，需由后端自动维护节点唯一性约束，防止脏数据复写。

3.4 系统的可扩展性与拓展演进方向

本系统的架构设计已预留了向更高阶 AI 应用演进的锚点：

（1）混合检索架构平滑升级：目前依赖纯图谱结构检索，未来可平滑接入向量数据库（如 Milvus 或 Faiss），实现“知识图谱结构化关系检索+文本语义向量空间检索”的双路混合检索召回。

（2）高级 RAG 技术融合：预留了引入 HyDE 算法的流程节点，以进一步处理极度口语化或模糊的用户提问，显著提升实体链接与意图匹配的准确率。

（3）多轮对话状态跟踪：系统设计考虑了扩展实现基于图谱子状态锁定的多轮对话记忆功能。例如，当用户在首轮确立了“高血压”的上下文环境后，后续追问“那如果我有肾病呢？”时，系统

能够在原图谱检索状态基础上进行多跳追加检索，而不丢失前置对话的历史实体约束。